

Re:Learn

Multimodal AI Models, Datasets, and System Implementation Plan

Purpose: Explain which models to build, what each model does, what data is needed, and how the remaining parts of the system will work. The plan is written for a practical student project, not a system trained from scratch.

1. What we are building

Re:Learn is an adaptive learning system. It should look beyond whether an answer is right or wrong and use the evidence available in a student's response to identify possible misconceptions, provide a focused explanation, and check understanding again.

Multimodal meaning: The system can accept more than one kind of input, such as typed text, a selected MCQ option, a photo of handwritten working, or a diagram. Different input types need suitable processing before the diagnosis model can use them. The core diagnosis model does not have to be one giant model that directly understands every format.

Recommended model plan

Part	What it is	Build approach
Model 1: Multimodal response diagnosis	Main model that predicts possible misconceptions from the question and student evidence.	Use a pretrained text model plus image/OCR processing; combine the extracted information for diagnosis.
Model 2: Sequence analysis	Your added feature: checks patterns across a student's ordered answers.	Start with rules and a simple GRU experiment if enough sequence data is available.
Other components	Intervention, reassessment, tracking, and evaluation.	Use a pretrained LLM, reviewed content, and normal application/database logic. They are not separate models you must train.

Important: The problem statement does not require a fixed number of models. This is a practical design. Model 1 supports the core diagnosis task; Model 2 is an additional sequence feature.

2. Model 1 — Multimodal Misconception Diagnosis

Purpose: Identify and differentiate possible misconceptions from the student's answer and any available reasoning, calculation steps, code, or image.

Model type and recommended base

Item	Recommendation
Model type	Supervised classification using a pretrained language model, with an image-to-text/vision processing path for visual responses.
Text base model	DeBERTa-v3-base for text classification. DistilBERT is a lighter alternative.
Image path	Use OCR/handwriting recognition to read written work. For diagrams or graphs, use a vision-language model or a task-specific image parser where needed.
How they work together	Extract text and structured visual details first. Send the question, extracted answer, working, and relevant context to the diagnosis classifier. Keep the original image for review when extraction is uncertain.

Why this design?

A single text classifier is not automatically able to read photos, handwriting, graphs, and diagrams. A modular design is easier to build and test: one path handles typed answers, another extracts information from images, and the diagnosis stage uses the combined evidence. This also lets the team replace an OCR or vision tool without rebuilding the whole system.

Flow

Student question + response → identify input type → extract text/numbers/diagram details if needed → combine with question and concept → diagnosis classifier → ranked possible misconceptions + confidence → accept, ask a follow-up, or mark uncertain.

Example use cases

Use case	Input	What the system should do
MCQ	Question and selected option	Use the selected option as evidence. Map it to likely misconceptions, but avoid treating an option as proof of a misconception.
Numerical	Question, final number, and optional steps/photo	Check the result and inspect formula choice, units, and calculation steps. A wrong final number alone may not reveal the cause.
Theory answer	Typed explanation	Analyze the wording and reasoning to identify the concept that may be misunderstood.
Handwritten solution	Photo of equations or written explanation	Read the handwriting, extract steps, and pass the extracted evidence to diagnosis. Ask for a clearer image if reading is unreliable.
Diagram or graph	Student-drawn force diagram, ray diagram, or graph	Extract labels, directions, axes, and relationships; use these as evidence for diagnosis.
Code, if included in your chosen domain	Code and question context	Parse the code and inspect logic errors. For a Class 10 Physics-only demo, this use case is not needed.

What if reasoning is missing?

- Use the available response, such as the selected option or numerical result, as limited evidence.
- Return more than one possible misconception when appropriate, with confidence or evidence indicators.
- If the evidence is weak or several causes look similar, ask a short follow-up question instead of forcing one label.

- Use earlier answers only as additional evidence; do not assume the student has the same misconception in every question.

3. Dataset for Model 1

Dataset type: Supervised, labeled response examples. The dataset should include correct, incorrect, partly correct, and unclear responses—not only wrong answers.

Field	What to store	Why it matters
Question ID and text	The full question and its identifier	Diagnosis depends on the question context.
Topic and concept	Chapter and specific concept	Helps distinguish similar errors in different topics.
Question type	MCQ, numerical, theory, graph, diagram, etc.	Tells the system how the response should be interpreted.
Correct answer / accepted solution	Answer key and valid methods	Provides a reference for checking the response.
Student response	Option, text, number, image reference, or extracted response	The evidence being analyzed.
Reasoning / working	Explanation, formula, steps, units, or diagram details when available	Often gives the clearest clue about the cause of an error.
Misconception label	Reviewed label, multiple labels if justified, or unknown/uncertain	The target the classifier learns to predict.
Evidence and review	Why the label applies; reviewer status	Reduces weak or unsupported labels.
Source and origin	Textbook/page, paper/question, real, authored, or synthetic	Makes examples traceable and prevents synthetic data being misrepresented.

How much data is needed?

Approximate amount	Use
100–200 labeled examples	Tiny proof of concept with only a few classes; likely unstable.
500–1,000 examples	Initial fine-tuning experiments and a first evaluation.
1,000–2,000 examples	Reasonable early target for a focused prototype with several misconception classes.
More examples per class	Needed for broader coverage, varied wording, and stronger testing.

These are planning estimates, not guaranteed minimums. Try to collect roughly 100–200 reviewed examples per important misconception class where possible. If you have 8 classes, for example, that suggests about 800–1,600 examples. A smaller dataset can still be useful for a demo, but its results must be described as preliminary.

Dataset sources

- Use one Class 10 Physics textbook as the main source for concepts, explanations, formulas, and chapter coverage.
- Use suitable exam papers and answer keys to create varied questions and expected answers, while checking reuse and licensing conditions.
- Create misconception examples with subject knowledge and review them. Textbooks usually do not provide enough real student wrong answers or reasoning by themselves.
- Keep real student responses, team-authored examples, and AI-generated synthetic examples clearly marked as different data origins.
- Split training and test data by question or source where possible. Do not put near-identical versions of the same question in both training and test sets.

4. Model 2 — Sequence-Based Misconception Detection

Purpose: Analyze a series of answers in order and find a pattern that may not be visible from one answer alone. This is your added feature, not a requirement to train a separate model in the problem statement.

Model type and recommended approach

Item	Recommendation
Model type	Sequence classification: a model reads ordered response features and predicts a sequence-level pattern or misconception.
First implementation	Start with transparent rules and the predictions from Model 1. This gives you a baseline and is easier to explain.
Model to experiment with	A small GRU (Gated Recurrent Unit), trained on response sequences. A GRU is an architecture, not a pretrained base model.
Why GRU	It is relatively small and suitable for short quiz sequences. It can learn order and repeated patterns without needing a large transformer.

Flow

Ordered quiz responses → Model 1 predictions and response features → sequence model/rules inspect related concepts and changes over time → recurring or connected pattern + confidence → diagnosis service combines this with individual evidence.

Example use cases

Use case	Example sequence	Possible result
Repeated misconception	Across three questions, student repeatedly says a force is needed to keep an object moving.	Evidence of a persistent force-and-motion misconception.
Connected concepts	Student confuses mass, weight, and acceleration in related questions.	Possible connected difficulty; ask a follow-up before making a strong claim.
Change over time	Student initially makes an error, then answers a new related question correctly after an explanation.	Evidence of improvement, to be confirmed by reassessment.
Same wrong result, different causes	Two students get the same numerical answer but use different formulas or reasoning across the quiz.	Use the reasoning and sequence context to distinguish possible causes.

Dataset for Model 2

Dataset type: Ordered learner-response sequences with a sequence-level label or reviewed pattern annotation.

Field	What to store
Anonymous learner ID and session ID	Group responses from the same learner and quiz without exposing identity.
Question order and question ID	Preserve the order and identify what each response tested.
Concept and response type	Provide context for relationships between questions.
Response and reasoning	Keep the original evidence, not just a correct/incorrect flag.
Model 1 output	Predicted misconception classes and confidence for each answer.
Sequence label	Reviewed pattern, such as recurring misconception, changing evidence, or unclear.
Intervention and reassessment history	Record what happened between answers when the sequence spans more than one learning step.

How much data is needed?

Approximate amount	Use
100–200 sequences	Test the format and build rule-based examples.
500–1,000 sequences	Initial GRU experiments; results may still be unstable.
1,000–3,000 diverse sequences	More useful prototype experiments if labels are reviewed and sequences vary.
More independent learners and patterns	Needed to test whether the model generalizes beyond repeated or synthetic examples.

Count independent sequences and learners, not only the total number of answers. Synthetic sequences can help test the pipeline, but they should not be presented as real learner evidence. Keep all sequences from the same learner in only one data split to avoid leakage.

5. How the Other Components Will Be Implemented

These are system components. They do not each need a separately trained model.

Component	How to implement it	Example
Diagnosis service	Normal backend logic combines Model 1 and optional sequence results, checks confidence, keeps evidence, and handles uncertain cases.	If two diagnoses are plausible, ask a follow-up rather than choosing blindly.
Intervention generation	Use a pretrained LLM with a prompt and reviewed Physics material. Retrieve relevant content (RAG) and ask the LLM to explain the diagnosed misconception simply.	For the belief that force maintains motion, explain a trolley moving on a low-friction surface and contrast force with change in motion.
Reassessment engine	Use a reviewed question bank and rules to select a new question for the same concept, preferably with a different wording or context.	After an explanation, ask what happens when net force is zero in a new situation.
Learner tracking	Store attempts, diagnoses, confidence, interventions, and reassessment outcomes in a database. Update status based on evidence over time.	Show a misconception as recurring, improving, unresolved, or not enough evidence.
Evaluation	Use scripts and held-out labeled data to calculate accuracy, precision, recall, macro F1, and performance by input type. Review false diagnoses.	Check whether the model handles handwritten numerical work as well as typed conceptual answers.

Suggested LLM

Start with a pretrained instruction model such as Qwen2.5-3B-Instruct for lighter local use, or Qwen2.5-7B-Instruct if your laptops can run it comfortably. You do not need to fine-tune this model for the first prototype. Use reviewed source material and constrain it to explain the diagnosed concept; check generated content for correctness.

Multimodal input handling

Input	Processing path
Typed text / MCQ	Pass the question, selected option, and text directly to the diagnosis stage.
Photo of handwritten work	OCR or handwriting recognition extracts text, numbers, equations, and steps. Show the extracted result for correction if uncertain.
Graph or diagram	Use a vision-language model or a specialized parser to identify labels, axes, arrows, and relationships. Keep a confidence/uncertainty signal.
Numerical answer	Use deterministic calculation checks for arithmetic and units where possible; use the model to interpret the student's method and likely misconception.

Do not claim that the system understands every image type until you test it. For a reliable demo, choose a small set of supported visual tasks—for example, handwritten equations and force diagrams—and clearly show what is and is not supported.

6. End-to-End Prototype Flow

This is the flow the website can demonstrate.

1. Choose a Physics chapter → **2. Attempt quiz questions** (MCQ, theory, numerical, and selected image-based responses) → **3. Process the response** (text directly; image through OCR/vision) → **4. Run individual diagnosis** → **5. Analyze answer sequence** if enough responses exist → **6. Combine evidence and confidence** → **7. Show a targeted explanation** → **8. Ask a fresh reassessment question** → **9. Update the learner profile.**

What the demonstration should prove

- Two students can give the same wrong answer but show different reasoning; the system can produce different likely diagnoses when the evidence supports it.
- A photo of a worked solution can be processed and used as evidence, demonstrating the multimodal path.
- Several related answers can reveal a recurring pattern that a single answer may not show.
- The intervention is linked to the diagnosis rather than being a generic explanation.
- A new question checks whether the student's understanding changed, and the learner record updates.

Evaluation targets

Set targets only after measuring a baseline. For an early prototype, you may use the following as goals, not claimed results:

Metric	Initial target or reporting plan
Individual diagnosis	Aim for 80% accuracy and macro F1 of 0.75 on a properly held-out, representative set; report per-class results too.
Sequence analysis	Compare against a simple rule-based baseline. Keep the GRU only if it improves held-out results.
Multimodal extraction	Report extraction success and diagnosis performance separately for typed text, handwriting, and diagrams.
Latency	Measure actual end-to-end time on your demo hardware; do not assume a fixed response time.
Intervention quality	Have a subject reviewer check factual correctness, relevance to the misconception, and clarity.
Reassessment	Report how often students answer the new question correctly, but do not treat one correct answer as proof of lasting learning.

7. Final Recommended Build Plan

Priority	Build	Reason
1	Create and review a focused Physics dataset with varied answers, reasoning, and some image-based examples.	Good labels and evidence are the foundation of diagnosis.
2	Build a simple baseline, then fine-tune one text diagnosis model.	Proves the core requirement before adding complexity.
3	Add OCR/image interpretation for a limited set of handwritten or diagram responses.	Makes the prototype multimodal in a testable way.
4	Add sequence rules and compare them with a small GRU experiment if data permits.	Demonstrates your added sequence feature without making it a blocker.
5	Integrate the LLM intervention, question-bank reassessment, and learner tracking.	Completes the adaptive loop around the diagnosis model.
6	Test each input type and the complete flow on held-out examples.	Shows what works and where the system is uncertain.

In one line: Train one main misconception diagnosis model, make its input multimodal through suitable text and image processing, optionally add a sequence model for your unique feature, and implement intervention, reassessment, tracking, and evaluation as connected system components—not as four more models to train.

Note: Model names, data quantities, and performance targets in this document are recommendations for planning and experimentation. They are not requirements explicitly set by the problem statement and do not guarantee a particular result.

Re:Learn — End-to-End Component Flow

How the components connect, in simple words, including which model is used at each step.

1. Student answers a quiz

The LMS collects the student's MCQ, written answer, numerical working, or uploaded image.

Used here: LMS and quiz logic — no AI model.

2. Input processing

For images, extract handwriting/text and interpret useful visual information such as diagrams or graphs. Prepare the result as text and structured features.

Used here: PaddleOCR for text extraction; a suitable vision-language model for visual understanding, if needed.

3. Individual misconception detection

Check one answer and its reasoning to predict possible misconception labels.

Used here: DeBERTa-v3-base text classifier. The backend sends it prepared text (for example, in a JSON API request) and receives predictions such as labels and confidence scores.

4. Sequence-based misconception detection

Look across the student's answers in question order to find repeated or connected misconception patterns.

Used here: Start with rule-based analysis; optionally train a GRU sequence model. This is Model 2.

5. Combine the diagnoses

Combine the individual prediction with the sequence-pattern result. If evidence is weak or unclear, ask a follow-up instead of making a confident diagnosis.

Used here: Backend/application logic and confidence thresholds — no separate model required.

6. Generate a personalized explanation

Use the likely misconception and approved learning material to create a targeted explanation.

Used here: Qwen2.5-7B-Instruct with RAG (retrieval from textbook and reviewed content).

7. Reassessment

Give a new question to check whether the student understood the concept.

Used here: Question bank and LMS logic — no AI model required initially.

8. Track learning progress

Save answers, predictions, explanations, and reassessment results so future learning can use that history.

Used here: Database and learner-tracking logic.

In one line: Student input → image/text processing → DeBERTa individual diagnosis → GRU or rules for sequence patterns → combine evidence → Qwen LLM explanation → reassessment → update learner profile.

Remember: DeBERTa asks, "What misconception might explain this answer?" The sequence model asks, "What pattern across answers suggests a recurring or connected misconception?" The vision component interprets images; it does not replace the misconception classifier. This is a proposed design, not a requirement to use these exact models.

Model A: Misconception Detection and Differentiation

Model A does more than mark an answer right or wrong. It predicts which misconception, if any, best explains an individual student's response.

Two parts of Model A's job

Detection	Identify whether the response suggests a misconception, a correct understanding, a calculation slip, a unit conversion error, or insufficient evidence.
Differentiation	Distinguish between different misconception classes that may lead to similar incorrect answers, using the question, student response, and any available reasoning or visual evidence.

Example

For a question about the relationship between current and resistance, a student might say that increasing resistance increases current. Model A can predict a relevant misconception label. If another student gives a wrong numerical result despite using the correct formula, Model A may instead predict a calculation slip. The exact diagnosis depends on the evidence and the quality of the training examples.

What Model A receives and returns

Input: The question and student's response/reasoning, plus text or structured information extracted from an image when available. The backend can send this to the deployed classifier through a JSON API request.

Output: One or more predicted labels with confidence scores. The application can use an uncertainty threshold to avoid forcing a diagnosis when evidence is weak.

How Model B differs

Model A diagnoses one response at a time. Model B analyzes the ordered series of a student's responses to identify repeated or connected patterns across questions. Model B does not replace Model A; it uses response history to add sequence-level evidence.

Dataset link: Your individual-response dataset has 900 records and 15 target classes. These classes support differentiation across misconception types as well as correct, calculation-slip, unit-conversion, and insufficient-evidence outcomes. This does not guarantee perfect differentiation; performance must be measured on held-out test data.